

Cubesat Space Protocol

CubeSat Space Protocol (CSP) is a small network-layer delivery protocol designed for CubeSats. The idea was developed by a group of students from Aalborg University in 2008, and further developed for the AAUSAT3 CubeSat mission that was launched in 2013. The protocol is based on a 32-bit header containing both network and transport layer information. Its implementation is designed for embedded systems such as the 8-bit AVR microprocessor and the 32-bit ARM and AVR from Atmel. The implementation is written in C and is ported to run on [FreeRTOS](#) and [POSIX](#) and pthreads-based operating systems such as [Linux](#). The three-letter acronym CSP was adopted as an abbreviation for *CAN Space Protocol* because the first MAC-layer driver was written for [CAN](#)-bus. The physical layer has since been extended to include several other technologies, and the name was therefore extended to the more general CubeSat Space Protocol without changing the abbreviation.

The protocol and the implementation is still actively maintained by Johan de Claville Christiansen, Space Inventor and GomSpace. The source code is available under an [LGPL](#) license and hosted on [GitHub](#).

Description

The CubeSat Space Protocol enables distributed embedded systems to deploy a service-oriented network topology. The layering of CSP corresponds to the same layers as the [TCP/IP](#) model. The implementation supports a connection oriented transport protocol (Layer 4), a router-core (Layer 3), and several network-interfaces (Layer 1–2). A service-oriented topology eases the design of satellite subsystems, since the communication bus itself is the *interface* to other subsystems. This means that each subsystem developer only needs to define a service-contract, and a set of port-numbers their system will be responding on. Furthermore, subsystem inter-dependencies are reduced, and redundancy is easily added by adding multiple similar nodes to the communication bus.

Key features include:

- Simple [API](#) similar to [Berkeley sockets](#).
- Router core with static routes. Supports transparent forwarding of packets over e.g. spacelink.
- Support for both connectionless operation (similar to [UDP](#)), and connection oriented operation (based on [RUDP](#)).
- Service handler that implements [ICMP](#)-like requests such as ping and buffer status.
- Support for loopback traffic. This can e.g. be used for [Inter-process communication](#) between subsystem tasks.
- Optional support for broadcast traffic if supported by the physical interface.
- Optional support for promiscuous mode if supported by the physical interface.
- Optional support for encrypted packets with [XTEA](#) in [CTR](#) mode.
- Optional support for [HMAC](#) authenticated packets with truncated [SHA-1](#) HMAC.

Operating systems supported

CSP should compile on all platforms that have a recent version of the [gcc](#) compiler. CSP requires support for [C99](#) features such as inline functions and designated initializers.

- [FreeRTOS](#) – Tested on [AVR8](#), [AVR32](#) and [ARM7](#).
- [Linux](#) – Tested on [x86](#), [x86-64](#) and [Blackfin](#).
- [Mac OS X](#)
- [Microsoft Windows](#)

Physical layer drivers

CSP supports several physical layer technologies. The LGPL licensed source code contains an implementation of a fragmenting CAN interface and drivers for SocketCAN and the Atmel AT90CAN128, AT91SAM7A1 and AT91SAM7A3 processors. From version 1.1 onwards, CSP also includes interfaces for [I2C](#) and [RS-232](#). Interfaces need only to implement a function to transmit a packet, and insert received packets into the protocol stack with the `csp_new_packet` function. CSP has been successfully tested on top of the following layers:

- [CAN](#)
- [I2C](#)
- [RS-232](#) using the [KISS \(TNC\)](#)^[1] protocol
- [CCSDS](#) 131.0-B-1-S^[2]/131.0-B-2^[3] space link protocol
- [UDP/IP](#)
- [SPI](#)^[4]

Protocol header

Version 1

The port range is divided into three adjustable segments. Ports 0 to 7 are used for general services such as ping and buffer status, and are implemented by the CSP service handler. The ports from 8 to 47 are used for subsystem specific services. All remaining ports, from 48 to 63, are ephemeral ports used for outgoing connections. The bits from 28 to 31 are used for marking packets with HMAC, XTEA encryption, RDP header and CRC32 checksum.

CubeSat Space Protocol

Original author(s)	Johan de Claville Christiansen
Developer(s)	SpaceInventor, GomSpace
Initial release	26 April 2010
Stable release	1.6 / April 17, 2020
Written in	C, Python
Operating system	FreeRTOS, Linux, Mac OS X, Microsoft Windows
Type	Protocol
License	GNU Lesser General Public License
Website	CubeSat Space Protocol Github Library (http://www.libcsp.org)

CSP Header 1.x																															
Bit offset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	
0	Priority		Source				Destination				Destination Port				Source Port				Reserved				H M A C		X T E A						
32	Data (0 – 65,535 bytes)																														

Version 2

CSP Header 2.x																																			
Bit offset	47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18					
0	Priority		Destination														Source																		
48	Data (0 – 65,535 bytes)																																		

References

1. "KISS Protocol" (<http://www.ax25.net/kiss.aspx>).
2. "TM Synchronization and Channel Coding CCSDS 131.0-B-1 Blue Book" (<https://public.ccsds.org/Pubs/131x0b1s.pdf>) (PDF). September 2003. Retrieved December 9, 2022.
3. "TM Synchronization and Channel Coding Recommended Standard CCSDS 131.0-B-2" (<https://public.ccsds.org/Pubs/131x0b2ec1s.pdf>) (PDF). August 2011. Retrieved December 9, 2022.
4. "Information Processing and Digital Communications in a Modular Satellite" (<https://www.researchgate.net/publication/357358305>).

External links

- Project Website and source code hosting (<http://www.libcsp.org/>)
- Aalborg University, Student Satellite Activities (<http://studentspace.aau.dk>)
- Aalborg University, AAUSAT3 – the first AAU cubesat to use CSP (<http://aausat3.space.aau.dk>)
- GomSpace ApS (<http://gomspace.com>)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Cubesat_Space_Protocol&oldid=1161865869"

▪